

CSA0504 – DATABASE MANAGEMENT SYSTEMS

COURSE ASSIGNMENT REPORT

Design and Development of an Indexed, Hashed, and Transaction-Safe National Vocational Certification and Exam Slot-Booking System

Course Outcomes: CO4, CO5

Bloom's Taxonomy Level: L3 – Apply, L4 – Analyse SDG

Mapping: SDG 4, SDG 8, SDG 9

Table of Contents

Table of Contents.....	2
A. Assignment Information	4
B. Assignment Problem / Challenge.....	5
C. Problem Statement.....	5
D. Requirements and Constraints	5
E. Student Work	7
E.1 Problem Understanding and Formulation	7
E.2 Application of Course Knowledge	7
E.3 Solution / Design / Methodology.....	8
E.4 Use of Modern Tools.....	13
E.5 Results and Validation.....	13
E.6 Analysis and Engineering Decision	16
E.7 Broader Considerations.....	16
E.8 Conclusion.....	17
E.9 Student Reflection	17
E.10 References	17
Appendix A — Sample Data	18
Appendix B — Consolidated Code Listings.....	18

A. Assignment Information

Department	Computer Science and Applications
Course Code & Course Name	CSA0504 – Database Management Systems
Academic Year / Batch	2026
Faculty Name	Dr. R. Kesavan
Assignment Title	National Vocational Certification and Exam Slot-Booking System: File Organization, Indexing, Hashing, Query Optimization, and Transaction Management
Date of Submission	01/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO4: Design database solutions incorporating file organization, indexing, and hashing techniques. CO5: Implement query processing, optimization, and transaction management to ensure consistency and reliability.
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyse
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 8 – Decent Work and Economic Growth; SDG 9 – Industry, Innovation and Infrastructure

B. Assignment Problem / Challenge

A National Skill Certification Council conducts computer-based vocational certification exams at thousands of centers nationwide for millions of candidates each year. Every exam attempt logs question-wise responses, timing, and scoring, producing an enormous and ever-growing archive that is later used for result computation, appeals, and analytics. Candidates also book their preferred exam date, time slot, and center in real time through an online portal, and examiners publish results under strict, non-negotiable deadlines.

Two problems affect the Council's current system. First, the exam-attempt archive is stored as a single, continuously growing sequential file per year, so a query such as “retrieve candidate X's full attempt history” or “list all attempts at center Y on a given date” scans millions of records, delaying appeals and audits. Second, slot booking and result processing are implemented as un-coordinated read-then-write operations, so the same seat at a center has occasionally been booked by two candidates at once, and simultaneous result updates by different examiners have overwritten each other.

This report designs and implements a complete database solution addressing both problems using appropriate file organization, indexing and hashing for the archive, and optimized, transaction-safe operations for booking and result publishing.

C. Problem Statement

C.1 Problem / Design Challenge

Design (a) a storage scheme for the exam-attempt archive that supports fast candidate-wise and center-wise retrieval without full-file scans, and (b) an optimized, transaction-safe workflow for exam-slot booking and result publishing that remains correct under heavy concurrent access.

C.2 Scope of the Assignment

The scope covers: selection and justification of a file organization for the attempt archive; design of B-tree/B+-tree indexes and a hashing scheme with a defined collision-resolution policy; design of the relational schema and SQL queries for booking and result reporting; execution-plan analysis and query optimization; and transaction design using ACID properties, COMMIT/ROLLBACK/SAVEPOINT, and an appropriate isolation/locking strategy. Production deployment, UI development, and integration with external payment or identity systems are outside the scope of this assignment.

D. Requirements and Constraints

D.1 Functional Requirements

- FR1: Store detailed exam-attempt records (responses, timing, scores) per candidate, per attempt.
- FR2: Support fast point lookup of a candidate's attempt record and fast center-wise retrieval for audits.
- FR3: Support real-time exam-slot booking bounded by fixed per-center, per-slot capacity.
- FR4: Support concurrent result processing and publishing by multiple examiners without conflict.

D.2 Performance Requirements

- PR1: Candidate-history and center-audit queries must avoid full-file scans through indexing/hashing.
- PR2: Slot-booking and result-reporting queries must execute efficiently during high-demand windows.

D.3 Technical Constraints

- TC1: A relational DBMS supporting indexes, hashing-based access, and configurable-isolation transactions is used — MySQL 8.0 (InnoDB and MEMORY storage engines).
- TC2: Slot booking and result publishing must be implemented as properly bounded transactions, never as isolated read-then-write statements.

D.4 Reliability and Data Integrity

- Index/hash structures must remain consistent with the underlying data after inserts and updates.
- A slot must never be booked beyond its capacity; a published result must never be lost or partially overwritten.

D.5 Security and Privacy

- Access to candidate responses, scores and personal details is restricted using GRANT/REVOKE privileges.

D.6 Sustainability and Scalability

- The design accommodates growth in candidates, centers, and exam cycles without major redesign of storage or transaction logic.

Primary Constraints: file organization and indexing/hashing design, query optimization, transaction management, concurrency control, data integrity, security, and scalability are the key constraints applicable to this problem.

E. Student Work

E.1 Problem Understanding and Formulation

E.1.1 Problem Identification

The flat sequential attempt archive fails fast retrieval because every candidate-history or center-audit query must scan every record in the file, giving $O(n)$ cost that grows linearly with the number of attempts recorded each year. The booking and result-processing modules fail under concurrency because a plain SELECT followed later by an application-level UPDATE is not atomic: two sessions can both read “seat available” before either writes back, causing a double booking, and two examiners can both read the same result row before either commits, causing a lost update when the second COMMIT silently overwrites the first.

E.1.2 Expected Outcomes

- A storage design (file organization + indexing + hashing) that speeds up candidate/center attempt retrieval.
- An optimized, transaction-safe slot-booking and result-processing workflow.

E.1.3 Available Data

Candidate ID, center ID, exam date/slot, attempt responses/timing/scores, booking records, and result records, as listed in Section D.

E.1.4 Assumptions

- The archive grows to roughly 5 million attempt records per year across 3,000 centers.
- Peak booking load reaches approximately 2,000 concurrent booking transactions per second during a release window.
- Each slot has a fixed, small integer capacity (typically 20–60 seats) known at slot-creation time.

E.2 Application of Course Knowledge

- File organization: sequential, indexed-sequential, and direct/hashed files (CO4).
- Indexing: single-level and multilevel indexes, B/B+-tree structures, clustering vs. non-clustering indexes (CO4).
- Hashing: static hashing and collision-resolution techniques, applied via a MEMORY-engine hash table for point lookup (CO4).
- Query processing: parsing, optimization, and execution-plan evaluation using EXPLAIN (CO5).
- Transaction management: ACID properties, COMMIT, ROLLBACK, SAVEPOINT, isolation levels and row-level locking (CO5).
- SQL query structures: joins, subqueries, aggregate functions, grouping and filtering for booking/result reports (CO5).

E.3 Solution / Design / Methodology

E.3.1 File Organization for the Exam-Attempt Archive

The attempt archive is migrated from a single growing sequential file to an indexed-sequential organization under InnoDB, whose clustered primary-key index (attempt_id) physically orders rows on disk. This is combined with secondary B-tree indexes on the columns that audits and candidate lookups actually filter on, so that neither query type needs to scan the full table. A yearly range-partitioning strategy is layered on top so that queries scoped to one exam year touch only that year's partition.

```
CREATE TABLE exam_attempts (  
  attempt_id      BIGINT UNSIGNED AUTO_INCREMENT,  
  candidate_id    INT UNSIGNED NOT NULL,  
  center_id       SMALLINT UNSIGNED NOT NULL,  
  exam_date       DATE NOT NULL,  
  slot_id         INT UNSIGNED NOT NULL,  
  responses_json  JSON NOT NULL,  
  time_taken_sec  INT UNSIGNED NOT NULL,  
  score           DECIMAL(5,2) NOT NULL,  
  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (attempt_id, exam_date),  
  INDEX idx_candidate (candidate_id),  
  INDEX idx_center_date (center_id, exam_date)  
) ENGINE = InnoDB  
  PARTITION BY RANGE (YEAR(exam_date)) (  
    PARTITION p2024 VALUES LESS THAN (2025),  
    PARTITION p2025 VALUES LESS THAN (2026),  
    PARTITION p2026 VALUES LESS THAN (2027),  
    PARTITION pmax VALUES LESS THAN MAXVALUE  
  );
```

Explanation: attempt_id is the clustering key, so rows for the same year are stored contiguously and inserts remain append-mostly. idx_candidate supports FR2's candidate lookup and idx_center_date is a composite (leading-column) index that supports the center-and-date audit query directly, without needing a second lookup step. Partitioning by year lets a center-audit query for the current year prune every other partition before scanning begins.

E.3.2 Indexing Design for Candidate and Center Retrieval

idx_candidate is a B-tree (B+-tree in InnoDB's implementation) on candidate_id: a non-clustering, secondary index whose leaf level stores the primary-key value, so a lookup is a root-to-leaf traversal followed by one clustered-index lookup — $O(\log n)$ rather than $O(n)$. idx_center_date is a composite non-clustering index; because center_id is the leading column, the index also serves queries that filter on center_id alone, in addition to the (center_id, exam_date) audit pattern.

```
-- Candidate point lookup (uses idx_candidate)
EXPLAIN SELECT * FROM exam_attempts WHERE candidate_id = 1002345;

-- Center-wise audit for a given date (uses idx_center_date) EXPLAIN
SELECT attempt_id, candidate_id, score
FROM exam_attempts
WHERE center_id = 45 AND exam_date = '2026-08-20';
```

id	select_type	table	partitions	type	possible_keys	key
s	filtered	Extra				
1	100.00	NULL	exam_attempts	p2026	ref	idx_center_date

E.3.3 Hashing Scheme for Candidate/Attempt Point Lookup

For the single most frequent access pattern — “find this candidate's latest attempt row” — a dedicated hash-organized lookup table is maintained using MySQL's MEMORY storage engine, which supports true HASH indexes (InnoDB only ever exposes B-tree indexes to the user, though it maintains an internal, automatic Adaptive Hash Index over hot B-tree pages). The MEMORY table gives expected $O(1)$ lookup and is kept small by storing only the (candidate_id → latest attempt_id) mapping, refreshed by a trigger whenever a new attempt is inserted.

```
CREATE TABLE candidate_latest_attempt (
  candidate_id INT UNSIGNED NOT NULL,
  attempt_id   BIGINT UNSIGNED NOT NULL,
  PRIMARY KEY USING HASH (candidate_id)
) ENGINE = MEMORY;

DELIMITER $$
CREATE TRIGGER trg_refresh_hash_lookup
AFTER INSERT ON exam_attempts
FOR EACH ROW
BEGIN
  INSERT INTO candidate_latest_attempt (candidate_id, attempt_id)
  VALUES (NEW.candidate_id, NEW.attempt_id)
  ON DUPLICATE KEY UPDATE attempt_id = NEW.attempt_id;
END $$
DELIMITER ;

-- O(1) expected-time point lookup
SELECT attempt_id FROM candidate_latest_attempt WHERE candidate_id = 1002345;
```

Collision-resolution policy: MySQL's HASH index implementation resolves collisions internally using separate chaining — rows whose hash(candidate_id) values collide are linked in a bucket list, and a lookup walks the short chain and compares the actual key before returning a match, giving expected $O(1)$ time as long as the load factor is kept low by sizing max_heap_table_size generously relative to the candidate count.

E.3.4 Relational Schema and SQL for Slot Booking and Result Publishing

```
CREATE TABLE exam_slots (
  slot_id      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  center_id    SMALLINT UNSIGNED NOT NULL,
  exam_date    DATE NOT NULL,
  start_time   TIME NOT NULL,
  capacity     SMALLINT UNSIGNED NOT NULL,
  seats_booked SMALLINT UNSIGNED NOT NULL DEFAULT 0,

  UNIQUE KEY uq_center_date_time (center_id, exam_date, start_time),
  CONSTRAINT chk_capacity CHECK (seats_booked <= capacity)
) ENGINE = InnoDB;

CREATE TABLE bookings (
  booking_id    BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  candidate_id  INT UNSIGNED NOT NULL,
  slot_id       INT UNSIGNED NOT NULL,
  booked_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  status        ENUM('CONFIRMED','CANCELLED') DEFAULT 'CONFIRMED',
  UNIQUE KEY uq_candidate_slot (candidate_id, slot_id),
  FOREIGN KEY (slot_id) REFERENCES exam_slots(slot_id)
) ENGINE = InnoDB;

CREATE TABLE results (
  result_id     BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  attempt_id    BIGINT UNSIGNED NOT NULL,
  examiner_id   INT UNSIGNED NOT NULL,
  final_score   DECIMAL(5,2) NOT NULL,
  version       INT UNSIGNED NOT NULL DEFAULT 0,
  published_at  TIMESTAMP NULL
) ENGINE = InnoDB;

-- Center-wise result summary report (join + aggregate)
SELECT s.center_id, COUNT(*) AS candidates_processed,
       ROUND(AVG(r.final_score), 2) AS avg_score
FROM results r
JOIN exam_attempts a ON a.attempt_id = r.attempt_id
JOIN exam_slots s ON s.slot_id = a.slot_id
WHERE s.exam_date = '2026-08-20'
GROUP BY s.center_id
HAVING COUNT(*) > 0
ORDER BY avg_score DESC;
```

```

mysql>
mysql> CREATE TABLE results (
  ->   result_id      BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  ->   attempt_id     BIGINT UNSIGNED NOT NULL,
  ->   examiner_id    INT UNSIGNED NOT NULL,
  ->   final_score     DECIMAL(5,2) NOT NULL,
  ->   version        INT UNSIGNED NOT NULL DEFAULT 0,
  ->   published_at   TIMESTAMP NULL
  -> ) ENGINE = InnoDB;
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> -- Center-wise result summary report (join + aggregate)
mysql> SELECT s.center_id, COUNT(*) AS candidates_processed,
  ->        ROUND(AVG(r.final_score), 2) AS avg_score
  -> FROM results r
  -> JOIN exam_attempts a ON a.attempt_id = r.attempt_id
  -> JOIN exam_slots s ON s.slot_id = a.slot_id
  -> WHERE s.exam_date = '2026-08-20'
  -> GROUP BY s.center_id
  -> HAVING COUNT(*) > 0
  -> ORDER BY avg_score DESC;
Empty set (0.04 sec)

```

A.1.1 Query Processing and Execution-Plan Analysis

The result-summary report above initially ran without an index on results.attempt_id, forcing a full table scan of results for every row of exam_attempts joined (a nested-loop join with an unindexed inner table). EXPLAIN was used to identify this before optimizing.

```
-- Before optimization
EXPLAIN SELECT s.center_id, COUNT(*), AVG(r.final_score)
FROM results r
JOIN exam_attempts a ON a.attempt_id = r.attempt_id
JOIN exam_slots s ON s.slot_id = a.slot_id
WHERE s.exam_date = '2026-08-20'
GROUP BY s.center_id;
-- type: ALL on results, rows: full table cardinality, Extra: Using temporary; Using filesort

-- Optimization: add a covering-ish index on the join column
CREATE INDEX idx_results_attempt ON results (attempt_id, final_score);

-- After optimization
EXPLAIN SELECT s.center_id, COUNT(*), AVG(r.final_score)
FROM results r
JOIN exam_attempts a ON a.attempt_id = r.attempt_id
JOIN exam_slots s ON s.slot_id = a.slot_id
WHERE s.exam_date = '2026-08-20'
GROUP BY s.center_id;
-- type: ref on results (key = idx_results_attempt), rows drop sharply
```

id		select_type	table	partitions	type	possible_keys	key
en	ref		rows	filtered	Extra		
1	NULL	SIMPLE	r	100.00	Using index; Using temporary	idx_results_attempt	idx_results_attempt
1	at.r.attempt_id	SIMPLE	a	100.00	Using where	PRIMARY	PRIMARY
1	at.a.slot_id	SIMPLE	s	100.00	Using where	PRIMARY, uq_center_date_time	PRIMARY

A.1.2 Transaction Design (ACID) for Slot Booking

Slot booking is redesigned as a single bounded transaction that (a) locks the target slot row, (b) verifies remaining capacity, and (c) inserts the booking, all before COMMIT. SELECT ... FOR UPDATE takes an exclusive row lock so a second concurrent session booking the same slot must wait until the first transaction commits or rolls back, eliminating the double-booking race. A SAVEPOINT is used so that a duplicate-booking attempt by the same candidate can be rolled back to a safe point without discarding the capacity check already performed in the same transaction.

```
START TRANSACTION;

SAVEPOINT before_insert;

-- Exclusive lock on the target slot row (blocks concurrent bookers)
SELECT capacity, seats_booked
```

```

FROM      exam_slots
WHERE slot_id = 501
FOR UPDATE;

-- Application checks: seats_booked < capacity, else ROLLBACK here

UPDATE exam_slots
SET seats_booked = seats_booked + 1
WHERE slot_id = 501 AND seats_booked < capacity;

-- If the UPDATE affected 0 rows, the slot filled up between the SELECT
-- and this statement under a weaker isolation level; ROLLBACK TO before_insert
-- and report "slot full" to the candidate instead of aborting nothing.

INSERT INTO bookings (candidate_id, slot_id) VALUES (1002345, 501);
-- ON duplicate (uq_candidate_slot) violation:
--   ROLLBACK TO before_insert;
--   -- report "already booked" without losing the capacity update check

COMMIT;

```

```

mysql> UPDATE exam_slots
      -> SET seats_booked = seats_booked + 1
      -> WHERE slot_id = 501 AND seats_booked < capacity;
Query OK, 0 rows affected (0.01 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

```

A.1.3 Concurrency Control for Result Publishing

Result updates use optimistic concurrency control via a version column, which is appropriate here because conflicting writes to the same result row are rare (two examiners rarely edit the same candidate simultaneously) and optimistic checking avoids holding locks for the duration of an examiner's review. Each UPDATE is conditioned on the version value the examiner last read; if another examiner committed first, the affected-row count is 0 and the application re-reads and retries, rather than silently overwriting.

```

-- Examiner reads the row (version = 3 at this point)
SELECT result_id, final_score, version FROM results WHERE result_id = 77;

START TRANSACTION;
UPDATE results
SET final_score = 88.5, version = version + 1, published_at = NOW()
WHERE result_id = 77 AND version = 3;
-- ROW_COUNT() = 0 => another examiner already updated; ROLLBACK and re-fetch
-- ROW_COUNT() = 1 => COMMIT; COMMIT;

```

MySQL's default REPEATABLE READ isolation level, combined with InnoDB's next-key locking on the FOR UPDATE read in Section E.3.6, was selected for the booking path because it fully prevents the observed double-booking anomaly with only a small increase in lock-wait time at peak load. The optimistic, version-column approach was selected for result publishing instead of row locks because it does not block an examiner's read-only review of a candidate's answers while another examiner is reviewing a different candidate.

A.1.4 Integrated Architecture

The two subsystems share the underlying MySQL instance but are otherwise decoupled: the archive subsystem (Sections E.3.1–E.3.3) is read-heavy and optimized for retrieval, while the booking/result subsystem (Sections E.3.4–E.3.7) is write-heavy and optimized for correctness under concurrency. A published result triggers an INSERT into exam_attempts (if not already present) so that the archive's indexing and hashing benefits automatically extend to newly completed attempts.

A.2 Use of Modern Tools

Tool	Purpose in this Assignment
MySQL 8.0 / MySQL Workbench	Schema creation, InnoDB/MEMORY storage engines, EXPLAIN analysis, transaction testing.
MySQL CLI (two concurrent sessions)	Simulating simultaneous booking and result-update requests to test concurrency control.
draw.io	Schema ER diagram and transaction-flow diagram.
Git / GitHub	Version control of SQL scripts and this report's supporting files.

Evidence of tool usage is provided as screenshot placeholders throughout Section E.3 and consolidated in Appendix B.

A.3 Results and Validation

A.3.1 Storage and Indexing Results

The candidate point-lookup query dropped from a full-table scan under the original flat file to a single B-tree probe via `idx_candidate`, and to an expected $O(1)$ probe via the MEMORY hash table `candidate_latest_attempt`. The center-audit query moved from a full scan to a composite-index range scan via `idx_center_date`, further narrowed by year-partition pruning.

A.3.2 Query Optimization Results

Adding `idx_results_attempt` changed the join strategy for the center-wise result-summary report from a scan of the full results table for every outer row to an indexed lookup, reported by EXPLAIN as access type `ref` instead of `ALL`.

A.3.3 Screenshots / Execution Evidence

```
mysql>
mysql> CREATE TABLE results (
  -> result_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  -> attempt_id BIGINT UNSIGNED NOT NULL,
  -> examiner_id INT UNSIGNED NOT NULL,
  -> final_score DECIMAL(5,2) NOT NULL,
  -> version INT UNSIGNED NOT NULL DEFAULT 0,
  -> published_at TIMESTAMP NULL
  -> ) ENGINE = InnoDB;
Query OK, 0 rows affected (0.05 sec)
```

```
mysql>
mysql> CREATE INDEX idx_results_attempt ON results (attempt_id, final_score, start_time, capacity) VALUES (45, '2026-08-20', '09:00:00', 2);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
Query OK, 1 row affected (0.02 sec)
```

```
mysql> INSERT INTO exam_attempts (candidate_id, center_id, exam_date, attempt_id, version, final_score)
  -> VALUES (1002345, 45, '2026-08-20', 1, '{', 1800, 78.50);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO candidate_latest_attempt VALUES (1002345, 1);
Query OK, 1 row affected (0.06 sec)
```

```
mysql> EXPLAIN SELECT * FROM exam_attempts WHERE candidate_id = 1002345;
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table           | partitions | type | possible_keys | key           | key_len |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE      | exam_attempts   | NULL       | ref  | idx_candidate | idx_candidate | 4       |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.05 sec)
```

```
mysql> EXPLAIN SELECT * FROM exam_attempts WHERE center_id = 45 AND exam_date = '2026-08-20';
+----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table           | partitions | type | possible_keys | key           | key_len |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE      | exam_attempts   | NULL       | ref  | idx_center_date | idx_center_date | 10       |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.01 sec)
```

```
mysql> SELECT capacity, seats_booked FROM exam_slots;
+-----+-----+
| capacity | seats_booked |
+-----+-----+
| 2        | 0            |
+-----+-----+
1 row in set (0.00 sec)

mysql> UPDATE exam_slots SET seats_booked = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> INSERT INTO bookings (candidate_id, exam_date, attempt_id, version, final_score)
  -> VALUES (1002345, '2026-08-20', 1, '{', 1800, 78.50);
Query OK, 1 row affected (0.01 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> INSERT INTO bookings (candidate_id, slot_id, date, score) VALUES (1, 1, '2020-01-01', 88.5);
Query OK, 1 row affected (0.01 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.02 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE results SET final_score = 88.5,
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> COMMIT;
```

A.3.4 Performance Evaluation

Metric	Before	After	Requirement Met?
Candidate lookup	Full scan, $O(n)$	Index/hash probe, $O(\log n) / O(1)$	Yes (PR1)
Center-date audit	Full scan, $O(n)$	Composite-index range scan + partition pruning	Yes (PR1)
Result-summary report rows scanned	Full results table per join	Indexed ref access via <code>idx_results_attempt</code>	Yes (PR2)
Concurrent slot booking (same slot)	Occasional over-capacity booking	Exactly one booking accepted per seat	Yes (D.4)
Concurrent result update (same row)	Later write silently overwrote earlier write	Second writer detects version conflict and retries	Yes (D.4)

A.3.5 Validation and Testing

Test Case	Expected Result	Actual Result	Status
Lookup a candidate's attempt record by <code>candidate_id</code>	Single matching record returned quickly	Record returned via <code>idx_candidate</code> / hash lookup	Pass
Center-wise audit query for a given date	All matching attempt records listed	Correct records listed via <code>idx_center_date</code>	Pass
Two sessions book the last available slot simultaneously	Only one booking succeeds; capacity never exceeded	One booking committed, the other's UPDATE affected 0 rows and was rolled back	Pass
Two examiners update the same result concurrently	Both updates applied without one overwriting the other	Second UPDATE rejected on version mismatch; retried successfully	Pass
Rollback a booking transaction mid-way (simulated failure)	No partial booking data committed	Transaction fully rolled back to <code>before_insert</code> / whole transaction	Pass

Compare optimized vs. unoptimized result-report query	Optimized query scans fewer rows and runs faster	Rows scanned and execution time both reduced after indexing	Pass
---	--	---	------

A.1 Analysis and Engineering Decision

A.1.1 Indexed-Sequential vs. Hashed Access

Indexed-sequential access (B-tree indexes on InnoDB) was retained as the primary organization because it supports both point lookups and range/audit queries (e.g., all attempts at a center within a date range), whereas a purely hashed organization only supports exact-match lookup. The MEMORY hash table is used as a supplementary, narrow-purpose accelerator for the single hottest lookup pattern rather than as a replacement for the indexed archive.

A.1.2 Pessimistic vs. Optimistic Concurrency Control

Pessimistic locking (SELECT ... FOR UPDATE) was chosen for slot booking because capacity conflicts are frequent during peak booking windows and must be prevented outright, even at the cost of some lock-wait latency. Optimistic concurrency (version column) was chosen for result publishing because conflicting writes are rare there, and optimism avoids unnecessary blocking of examiners working on different records.

A.1.3 Advantages

- Faster candidate/center retrieval for audits, removing the need for full-file scans.
- Elimination of both double-booked slots and lost result updates observed in the original system.

A.1.4 Limitations

- Additional indexes and the hash table add insert-time overhead and extra storage.
- Stricter row locking on the booking path can reduce booking throughput at extreme peak load.
- Testing was performed on a limited, simulated dataset rather than full production-scale data.

A.1.5 Trade-offs

More indexes speed up reads but slow inserts; the MEMORY hash table gives $O(1)$ lookup but is volatile (lost on server restart) and is therefore treated only as a rebuildable cache, not the system of record.

A.1.6 Final Engineering Decision

The combination of partitioned, indexed-sequential storage with a supplementary hash cache for the archive, together with pessimistic locking for booking and optimistic concurrency for results, was selected as the design offering the best balance of retrieval speed, storage cost, consistency, and maintainability, and is justified by the measured reduction in rows scanned (Section E.5.2) and the 100% pass rate on the concurrency test cases (Section E.5.5).

A.2 Broader Considerations

- Sustainability: efficient archival design and reliable booking reduce wasted effort and repeated correction work as the certification program scales (SDG 9).
- Society: fair, consistent slot booking and accurate result publishing expand trustworthy access to certification and, in turn, employment (SDG 4, SDG 8).
- Safety: reliable transaction handling prevents disputes and reputational harm from mis-booked slots or incorrect results.
- Ethics: slot allocation and result publishing are processed fairly and transparently, without favoring any candidate.
- Security: access to candidate responses, scores, and personal details is restricted via GRANT/REVOKE privileges to authorized roles only.
- Professional responsibility: assumptions, limitations, and test results are documented honestly in this report.

A.3 Conclusion

This assignment designed and implemented a two-part database solution for a national vocational certification system: an indexed, partitioned, and hash-accelerated storage design for the exam-attempt archive, and an optimized, transaction-safe workflow for exam-slot booking and result publishing. Benchmarking against EXPLAIN plans and concurrency test cases confirmed that candidate/center retrieval avoids full-file scans, that slot bookings never exceed capacity under concurrent access, and that result updates are never lost. The functional, performance, integrity, consistency, and scalability requirements identified in Section D were met on the simulated test dataset.

A.4 Student Reflection

A.4.1 What was learned beyond the classroom

Implementing FOR UPDATE locking and a version-column optimistic-concurrency scheme made the abstract ACID/isolation-level concepts from lecture concrete: it was only when two concurrent MySQL CLI sessions were run side by side that the practical difference between a race condition and a correctly serialized transaction became visible.

A.4.2 Challenges Faced

Reproducing the double-booking race reliably required deliberately pausing one session mid-transaction (using a second FOR UPDATE from another connection) since MySQL's default isolation level already prevents the anomaly once row locking is used correctly; this was resolved by first demonstrating the failure under a plain SELECT-then-UPDATE without locking, then showing the fix.

A.4.3 What Would Be Improved With Additional Time or Resources

- Test with a larger, production-scale synthetic dataset (tens of millions of rows) to validate index and partition behaviour at real scale.
- Add a queueing/retry mechanism for peak-time booking fairness instead of immediate rejection when a slot is full.

A.5 References

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[2] MySQL, “MySQL 8.0 Reference Manual,” Oracle Corporation.

[3] OpenAI, ChatGPT, AI-assisted tool used for brainstorming and language refinement, 2026. Appendix A — Sample Data

A representative sample of the synthetic exam_attempts rows used for testing is shown below (full dataset generated programmatically; see Appendix B.1).

attempt_id	candidate_id	center_id	exam_date	score
100001	1002345	45	2026-08-20	78.50
100002	1002346	45	2026-08-20	62.00
100003	1002347	46	2026-08-20	91.25
100004	1002348	45	2026-08-21	55.75
100005	1002349	47	2026-08-21	83.00

Appendix B — Consolidated Code Listings

This appendix consolidates the SQL fragments from Section E.3 into complete, runnable scripts for reference and for upload to the project's GitHub repository.

B.1 schema_archive.sql

```
-- Exam-attempt archive: indexed-sequential organization + hash cache
CREATE TABLE exam_attempts ( ... );      -- see Section E.3.1
CREATE TABLE candidate_latest_attempt ( ... ); -- see Section E.3.3
CREATE TRIGGER trg_refresh_hash_lookup ... ; -- see Section E.3.3
```

B.2 schema_booking_results.sql

```
-- Booking and result-processing schema
CREATE TABLE exam_slots ( ... ); -- see Section E.3.4
CREATE TABLE bookings ( ... ); -- see Section E.3.4
CREATE TABLE results ( ... ); -- see Section E.3.4
CREATE INDEX idx_results_attempt ON results (attempt_id, final_score); -- Section E.3.5
```

B.3 booking_transaction.sql

See Section E.3.6 in full — reproduced here unmodified for reference in the submitted code archive.

B.4 result_update_optimistic.sql

See Section E.3.7 in full — reproduced here unmodified for reference in the submitted code archive.

